National University of Singapore

School of Computing

Department of Computer Science

2024/2025

CP4101 B.Comp Dissertation

Linking Rust Code with Lean

Filbert Phang Kong San

Project ID: H258040

Supervisor: A/P Ilya Sergey

Main Evaluator: A/P Yu Haifeng

Abstract

Consensus protocols are at the heart of many modern distributed systems applications. Ensuring that a protocol is implemented correctly is important to ensure the reliability of distributed systems. Formal verification techniques have been applied to consensus protocol specifications, with recent work involving the verification of protocols using interactive theorem provers. However, there are performance and reliability concerns when extracting such implementations to more practical programming languages. This paper explores a novel approach to reconcile the trade-off between performance and correctness in verified consensus protocol implementations. Namely, we run the verified consensus protocol implementation in a theorem prover directly, then we write a shim layer in a more performant language to wrap over the theorem prover implementation. The result is a program that can be easily integrated into client applications due to the choice of language, while still remaining verifiably correct where it matters. We then build a sample application with this shim called Bythors, which is a replicated key-value store in Rust that utilises an implementation of the Raft consensus algorithm in Lean 4. We benchmark Bythors it to demonstrate that this approach achieves performance competitive with other replicated key-value stores.

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, A/P Ilya Sergey, for his invaluable guidance, support, and patience throughout the course of this project. His insight and expertise have been instrumental in shaping my understanding and approach. I also extend my thanks to his PhD student, George Pîrlea, for his helpful feedback, patience, and willingness to discuss ideas and challenges along the way.

Contents

Chapter 1

Introduction

Consensus is the problem of requiring multiple different processes to agree on a single value [21]. The consensus problem is at the heart of many distributed systems, including distributed databases, blockchain, and cloud computing. Consensus protocols are algorithms that enable different processes to communicate in a known manner such that consensus may be achieved within the network.

Given their importance in real-world applications, implementations of consensus protocols should strive for correctness as far as possible, as any bugs in the implementation may lead to catastrophic outcomes [23]. Formal verification of consensus protocols has been useful in verifying their correctness, but often comes at the cost of scalability [29]. Interactive theorem provers have bridged this gap somewhat by allowing formal proofs to be written in the same language as the implementation code [33], but scalability still ultimately depends on the overall project architecture. However, implementations written in an interactive theorem prover are often unfit for use in practical settings due to performance limitations and lack of developer support.

Contributions. This paper explores a novel approach to reconcile the trade-off between performance and correctness in verified consensus protocol implementations. Namely, we explore running the verified consensus protocol implementation in a theorem prover directly. Then, we write a shim layer in a more performant language that is responsible for networking, handling client connections, and exposing the protocol interface to the rest of the application. The shim layer forwards any protocol events directly to the protocol implementation running in the theorem prover. The result is a program that can be easily integrated into client applications due to the choice of language, while still remaining verifiably correct where it matters.

We then build a sample application with this shim called Bythors, which is a replicated key-value store in Rust that utilises an implementation of the Raft consensus

algorithm in Lean 4, and benchmark it to demonstrate that Bythors achieves performance competitive with other replicated key-value stores.

Chapter 2

Background

We now describe, in greater detail, the background and context surrounding this project.

2.1 Distributed Systems

2.1.1 Introduction

Distributed computing refer to multiple distinct computational entities (or nodes) that work together as part of a larger system [21]. Distributed computing systems are used in many critical infrastructures today, including cloud services, blockchain platforms, and large-scale data processing frameworks like MapReduce and Apache Spark. Programs for distributed systems are notoriously difficult to write correctly due to having to deal with challenges like partial failures, data consistency, and data synchronisation [39].

2.1.2 Consensus

Consensus is a fundamental problem in distributed computing, where a group of nodes must agree on a single common value [21]. A consensus protocol is a mechanism for all nodes to communicate in a known manner such that consensus may be achieved within a network.

Consensus is essential in enabling techniques like state machine replication (SMR), where a single service is replicated to achieve greater redundancy and throughput [38]. In this case, consensus is required to ensure that the internal state of the service remains consistent across all its replicas.

Desirable properties of a consensus protocol include safety (nodes will never agree on different values), liveness (nodes eventually reach a consensus), fault tolerance (nodes can still reach consensus even if some are experiencing failures), and efficiency under realistic network conditions [42]. Given the importance of consensus protocols in

real-world applications, it would be sensible to formally verify implementations of such protocols to ensure that the desired properties hold under reasonable assumptions [23].

2.2 Formal Verification

2.2.1 Introduction

Formal verification is a method used to mathematically prove that a given system meets certain specifications or fulfills certain properties. This is done by modelling the system as mathematical structures so that the required guarantees can be proven on the model [22]. Formal verification is an important technique to ensure correctness in mission-critical situations. One example is the CompCert C compiler, which guarantees that machine code generated by the compiler is effectively equivalent to the original source code [31].

However, formal verification of software can be complex and expensive [29]. An additional problem is ensuring that the mathematical model created is a faithful representation of the original system, otherwise, the proven properties on the modelled system may not hold on the actual system. Likewise, the veracity of the proofs used is also crucial: if an error exists in the proof of a certain property, we cannot actually guarantee that such a property holds for the original system.

Another limitation of formal verification techniques is scalability: the proven properties may only hold for a particular version of the specification. In other words, depending on the way the system is modelled, making minor changes to the specification (such as adding new features to the software) may require major rewrites of the proofs [27], which costs additional development resources.

2.2.2 Interactive Theorem Provers

Interactive theorem provers (also known as proof assistants) are tools that assist users to construct formal proofs in a step-by-step manner, verifying their proof against a set of logical rules backed by some form of higher-order logic foundation [33]. This allows users to write proofs that can be computer-verified to be correct, improving confidence in their formally verified system.

Interactive theorem provers may vary in their underlying logical foundations. For example, Coq[1] and Lean use the Calculus of Inductive Constructions [30, 35], Agda re-

[1]Coq has recently undergone a name change to Rocq. However, we shall continue to refer to it as Coq for the purposes of this report.

lies on an extension of Martin-Löf type theory [18], while Idris is based on Quantitative Type Theory [4]. Despite being a relatively recent development, interactive theorem provers have already been used to successfuly verify notable mathematical results, such as a proof for the Four-Color Theorem [26] in Coq, a proof that $BB(5) = 47176870$ in Coq [12], and a proof of the Polynomial Freiman-Rusza Conjecture in Lean 4 [7].

Interactive theorem provers have also been a major step forward in the formal verification of software. Namely, most interactive theorem provers are also programming languages. Thus, software implementation and proof can be written directly in the theorem prover itself, eliminating the need to first model the implementation before proving properties on it [16]. Since the proof is verified by the computer, users can guarantee that their proofs are correct as long as it compiles.

2.2.3 Program Extraction

Though programs can be written directly in the theorem provers themselves, this may not be desirable for practical purposes. Interactive theorem provers are fairly niche and often do not have the robust package ecosystems seen in more mainstream programming languages. As such, developer support for practical concerns like networking, file system I/O, and asynchronous programming may be lacking. In addition, programs written in interactive theorem provers may not be as performant as their counterparts written in mainstream languages.

To bridge this gap in practicality, some interactive theorem provers support converting their programs into a different programming language, in a process known as program extraction. This conversion also has the added benefit of allowing the extracted code to integrate with an existing code base. For example, the Coq proof assistant allows program extraction, where any structures or functions modelled in Coq can be automatically transformed into equivalent programs in more popular languages like OCaml, Haskell or Scheme [32]. The Idris theorem prover also includes various code generators, allowing programs written in Idris to be compiled to targets like C, JavaScript, Java, and Python [3].

One limitation of extraction is that even if the original implementation is verified, in order to trust that the extracted or generated code is correct, we must also trust that the code extractor or generator itself is correct [33]. This increases the amount of code that must be trusted, which results in a larger surface area for bugs to occur. For example, the extraction mechanism for Coq is implemented separately from the compiler and thus may be subject to different standards of code quality and correctness [1].

In addition, though optimisations are often included as part of the extraction process, the generated code may be difficult to read and understand by human developers, which may make it challenging to implement further performance optimisations. For example, Coq-extracted OCaml code often include liberal use of `Obj.magic`, a type-casting function which is normally avoided among OCaml developers for safety reasons [24]. However, such cases are permissible here as Coq has a stronger type system than OCaml. Nevertheless, the rationale for such "unsafe" casts may not be clear to the programmer purely from reading the extracted code.

2.2.4 Direct Interaction

Given the limitations of program extraction, we opt to approach the problem in a different way. Instead of extracting a working program from a protocol implemented in a theorem prover, we opt to run the protocol directly in the theorem prover itself. We then use external means like a Foreign Function Interface (FFI) to interact with the protocol, such as calling packet handlers or sending heartbeats. We describe in greater detail how this is done in Bythors in section 3.4.1.

Regardless of whether we extract a program or run it directly, we still need to trust the compiler to type-check the code to ensure that both the protocol implementation and its associated proofs are sound. However, when running directly in a theorem prover, we gain the advantage of not having to trust the extractor/code generator — only the compiler. In such situations, needing less trust is necessarily a good thing as having fewer moving parts means that we also have fewer places where bugs can occur.

2.3 The Raft Consensus Algorithm

2.3.1 Introduction

Raft is a consensus protocol that has gained wide adoption due to its simplicity and relative ease of implementation while still maintaining strong safety guarantees, as compared to alternatives like Paxos [37]. Raft is designed primarily to be used in the context of state machine replication.

2.3.2 Mechanism

The operation of Raft is split into consecutive blocks of time known as *terms*. Each term begins with an election, where nodes elect a leader node to service client requests. Raft guarantees that at most one leader node is active at any given point in time; if no leader is elected in an election, a new term is started and another election is carried

out. Normal operation does not begin until a leader is elected.

After a node is elected, that node acts as the leader for the rest of the term. The leader node is responsible for servicing client requests, and acts as the single source of truth for the system. The leader node also sends periodic heartbeats to follower nodes. If a follower node does not receive a heartbeat within a fixed interval, they declare that the leader is absent and begins a new term, where an election is held to decide a new leader.

Notably, the original specification of the Raft does not guarantee liveness under all conditions [41]; there was a major Cloudflare service degradation in 2020 where an omission fault resulted in a Raft cluster being unable to elect a leader [11, 28]. However, there are variants of Raft that solve that problem [20].

2.3.3 Implementations

The etcd Raft library is one of the most popular implementations of the Raft consensus algorithm, and serves many production clusters daily [36]. Despite being battle-tested, this implementation is not verified, and bugs have been found as recently as 2024 [17].

Several projects have provided a verifiably-correct implementation of Raft. For example, Verdi-Raft implements and verify Raft in the Coq proof assistant, which can be extracted to OCaml [44]. also, Taube et al. provide and verify an implementation of Raft in Ivy [40], which can be compiled to C++ [34]. We evaluate our project against these related works in chapter 4.

Chapter 3

Implementation Details

This section describes the overall architecture of our project while also documenting several key design decisions made. We first present a diagram illustrating the structure of our project (Figure 3.1). We then describe each component in greater detail below.

The source code of this project is available on GitHub at https://github.com/filbertphang/bythors.

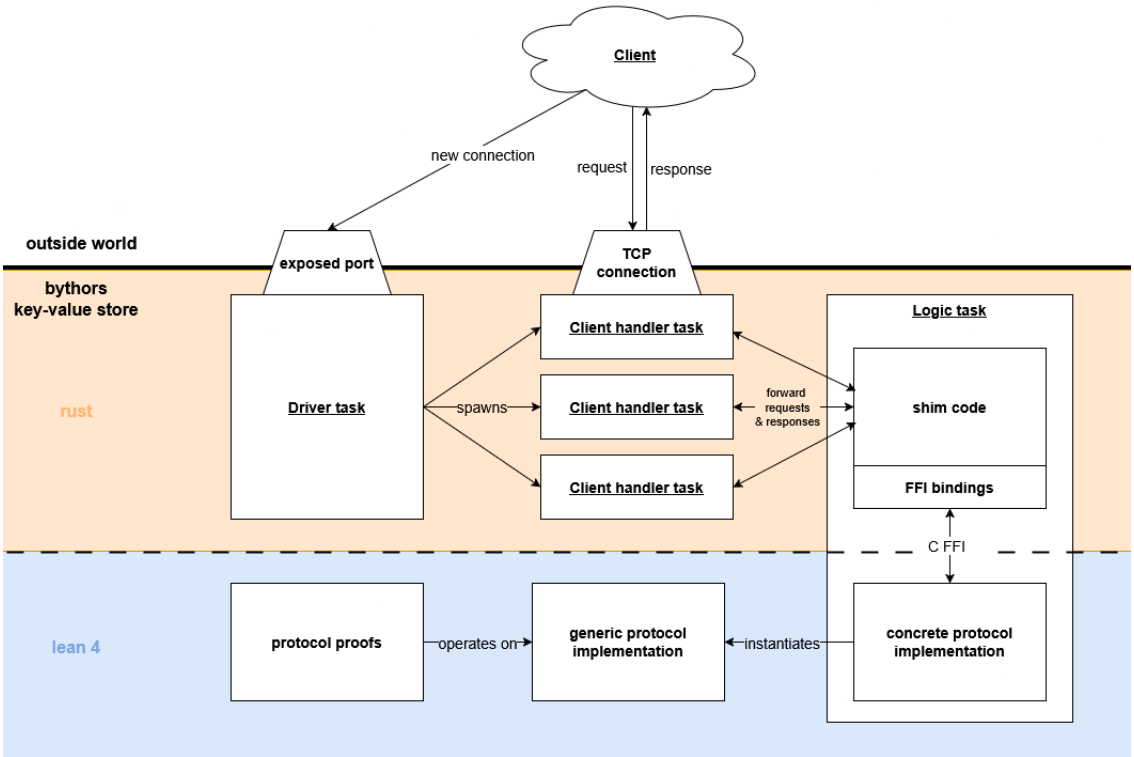

Figure 3.1: Overall structure of our project.

3.1 Overview

The Bythors key-value store is a replicated key-value store that is implemented using Rust and Lean 4. The components written in Lean are responsible for the logic or operation of the Raft consensus algorithm (section 3.2). The components written in Rust are responsible for managing node state, file system I/O, network communication (both with clients and with other nodes), and generally driving the protocol forward (section 3.3). Interfacing between the Lean and the Rust components is done via the C Foreign Function Interface (section 3.4).

3.2 Lean 4 Components

3.2.1 Introduction

Lean 4 is a state-of-the-art theorem prover and pure functional programming language with a dependent type system. We chose to implement Raft in Lean 4 for two main reasons: developer experience and performance.

For the former, on top of having full Visual Studio Code support via an extension [13], Lean 4 also provides helpful language features such as type classes, type inference, and metaprogramming with macros [6]. For the latter, Lean 4 is built with performance in mind: Lean 4 code compiles down to fast C code, and many primitives in Lean already have a C backend [35]. This allows us to efficiently interact with the Lean runtime using the C Foreign Function Interface (section 3.4.1), as we do not incur the overhead of setting up function calls according to the calling conventions.

3.2.2 Protocol Implementation

We first implement the Raft protocol in Lean 4 using generic types (`lib/Raft.lean`). In other words, this version of the protocol is parametrised over the types of node addresses, message payload, state machine, and others. Our implementation of Raft is effectively a Lean 4 translation of the Verdi-Raft implementation, originally written in Coq.

As an aside, we note that Lean is a pure functional programming language, all functions are pure. Hence, this protocol implementation is entirely event-driven, i.e., we can only specify what happens when a particular event occurs. Thus, an external driver (Section 3.3.4) must supply real-world events to advance the protocol state, since pure functions cannot trigger actions on their own.

We then instantiate the generic implementation with the concrete types that we will be using for our store (`lib/RaftConcrete.lean`). This is required as we cannot pass type parameters from the Rust side, so all type parameters must be instantiated in advance. Since we are using the Raft implementation in the context of a replicated key-value store, we also instantiate the state machine type as a hash map over string keys and values.

3.2.3 Protocol Verification

We note that we do not actually provide proofs of safety properties of the Raft implementation. While this may sound contradictory, the main objective of this project is to demonstrate that it is practical to interact directly (via FFI) with a consensus protocol implemented in a theorem prover, which would facilitate verification of the protocol. Notably, the presence (or absence) of the actual proofs do not impact performance as the verification only happens at compile-time and has no runtime impact.

Furthermore, Verdi-Raft (the project which our implementation was translated from) does contain proofs verifying the various guarantees of Raft [44]. As such, it is also possible for these proofs to be translated into Lean in a similar fashion. However, this would likely be a non-trivial task as it would also involve migrating the Verdi library that the above correctness proofs depend on. The alternative would be to re-engineer the required proofs by ourselves. In either case, writing the protocol proofs would go beyond the scope of this project.

3.3 Rust Components

3.3.1 Introduction

Rust is a systems programming language created at Mozilla. The language's main advantages are performance, type safety, and concurrency, which is achieved in no small part due to its memory model [19]. Rust is not garbage-collected, instead, it uses an ownership model to keep track of the lifetimes of variables, allowing the runtime to free the memory used by variables as soon as their lifetime ends. This ownership system is also responsible for preventing data races by allowing a variable to either be referenced by multiple readers or mutated by a single writer at any given point in time. Most importantly, this ownership system is embedded in its type system, which is in turn enforced by Rust's compiler. As a result, these checks are done at compile-time and do not contribute to runtime bloat, allowing programs written in Rust to be both safe and fast.

In addition, Rust also has a well-established package ecosystem in crates.io. Notably, Rust has several battle-tested networking and asynchronous programming crates such as tokio [9] and libp2p [25], which we find valuable for our project.

The components implemented in Rust include the network task (section 3.3.2), client handler tasks (section 3.3.3), and the logic task (section 3.3.4). Our project uses the tokio runtime to handle asynchronous programming and networking. Each "task" here refers to a `tokio::task`, which is defined to be a "light weight, non-blocking unit of computation" [10]. Tasks are managed by the tokio runtime, and may or may not be scheduled on different threads depending on thread availability and runtime configuration.

3.3.2 Network Task

The network task is responsible for accepting connections from new clients (`src/store/server.rs:250`). This is done by exposing a (configurable) port and listening on that port for new TCP connections from clients. Once a client connection is accepted, a client handler task is spawned to receive and respond to requests from that client via the TCP stream.

3.3.3 Client Handler Tasks

A client handler task is specific to a client, and is responsible for receiving requests and returning responses to that client (`src/store/server.rs:173`). Upon receiving a client request, the task first parses the stream of bytes into a suitable command (e.g. a `GET` or a `PUT` command for the key-value store), then forwards it to the logic task for processing. Once the client handler task receives the output of running that command from the logic task, it then responds by writing that output back to the client via the TCP stream.

3.3.4 Logic Task

The logic task is responsible for managing node state and driving the protocol forward by triggering protocol events such as sending heartbeats and forwarding user requests (`src/store/server.rs:58`). The logic task also handles intra-network communication, i.e., it sends and receives messages from other nodes in the distributed system. Intra-network communication is done using the libp2p crate.

The general event loop of the logic task is as follows: upon receiving a user request from a client handler task, the request information must first be transformed (or mar-

shalled) into a representation understood by Lean. Then, functions to handle protocol events in Lean are called from Rust via the C Foreign Function Interface (FFI), to advance the protocol state. The result is then marshalled back into a Rust representation, before being passed back to the client handler task.

3.4 Lean-Rust Interoperability

3.4.1 C Foreign Function Interface (FFI)

Recall that our project hinges on the decision to run the Raft protocol implementation in the theorem prover directly. This decision works particularly well with our choice of theorem prover (Lean 4): Lean is implemented partially in C++[1], and the Lean compiler itself uses C as an intermediate representation (IR) [35], which allows our implemented protocol to benefit from further optimisations done by C compilers like gcc or clang. Thus, compiled Lean code remains fairly performant, despite having the additional overhead of the Lean runtime.

In addition, the C IR also enables Lean to export functions as C bindings to be used by external libraries [5]. Since Rust also supports calling external C functions [8], we can export our protocol event handler Lean functions as C bindings, then perform an FFI call to those C bindings in our application code in Rust as long as we dynamically link the compiled protocol implementation library to our compiled application binary. This mechanism is the primary means in which we achieve interoperation between the protocol (implemented in Lean) and the application (implemented in Rust).

3.4.2 Data marshalling

However, Lean and Rust represent data differently, so to call functions across the language boundary, we must supply parameter data written in a format supported by the other language. This requires us to marshall data between the Lean and Rust representations.

We use the lean-sys crate to interact with the Lean runtime from Rust, such as by directly creating or manipulating Lean objects [14]. However, the last updated version of lean-sys only supports up to Lean v4.8.0[2], while our project uses Lean v4.11.0. To remedy this, we created a custom fork of lean-sys that supports up to Lean v4.12.0, the latest version of Lean (at the time of writing) [15].

[1] And partially in Lean itself.

[2] It should be noted that lean-sys has since been updated to support Lean v4.16.0.

We also implement helper functions to marshall between commonly used types such as strings, arrays, and tuples. We then build up towards the direct marshalling of complex structures like our data packets, entries, and messages.

Interacting with the Lean runtime from Rust posed a challenge. Since Lean uses a reference-counted memory model, we are given the responsibility of incrementing or decrementing the reference count of a Lean object whenever we manipulate it from Rust [5], and required careful implementation to ensure that we do not cause undefined behaviour. More importantly, as the memory for Lean objects are managed outside of the Rust runtime, the Rust borrow checker is unable to help us catch such memory-related errors, leading to many segmentation faults during the development of this project.

Chapter 4

Results

This section evaluates the performance of Bythors relative to other related work.

4.1 Methodology

To determine if running the protocol in a theorem-prover language is practically feasible, we ran performance benchmarks on Bythors and similar projects. Namely, we compare against the following projects:

- **etcd**, one of the most popular replicated key-value stores [2]
- **vard**, a replicated key-value store built on top of the verified Verdi-Raft protocol implementation [43]
- **Ivy-Raft**[1], a replicated key-value store built on top of a Raft protocol implemented and verified in Ivy [40]

Naturally, all the above projects are replicated key-value stores that utilise state-machine replication for fault tolerance, and use some implementation of Raft to achieve consensus over state machine state. We include the executables for each project in the project repository for convenience (`bench/bin`).

To benchmark each project, we first initialised each store locally with three nodes and allowed the initial election phase to complete. We then spawn 8 threads, each sending a series of `GET` or `PUT` requests until 100 total requests per thread are sent. The series of requests sent are randomised per thread, with equal probability of sending a `GET` or `PUT` request. Each `GET` or `PUT` request operates on a key uniformly selected from a set of 50. The benchmarking scripts for each key-value store can be found in `bench/scripts`.

[1]The actual key-value store implementation remains unnamed in the original paper, but for the purposes of this report, we refer to it as "Ivy-Raft".

4.2 Performance Benchmarks

We present the benchmarking results in table 4.1, and make several observations. More details can be found at `bench/RESULTS.md`.

Key-Value Store	Total Time (s)	Throughput (req/s)
Bythors	5.276967	151.602235
etcd	2.168403	368.935126
vard	14.442923	55.390450
Ivy-Raft	0.108757	7355.847071

Table 4.1: Performance of SMR-based key-value stores

First, we note that Ivy-Raft is more than an order of magnitude faster than the other k-v stores, which seems like an anomaly. As it turns out, this is because the implementation of Ivy-Raft does not maintain persistent node state, and thus does not incur the overhead of accessing disk [40]. This represents an incorrect usage of the Raft protocol, as Raft requires some aspects of the node state to be persisted to disk so that nodes can properly recover from crashes [37]. Due to this, we conclude that benchmarking performance against Ivy-Raft will not make for a meaningful comparison.

Next, we note that the performance of Bythors remains somewhat competitive with etcd and vard. Naturally, we do not expect to outperform the industry-standard etcd which has seen years of performance optimisations to reach its current state. However, we observe an almost 3x speedup and throughput increase over vard, which is surprising given the lack of optimisation work done for Bythors.

We hypothesise that the difference in performance between Bythors and vard can be attributed to the choice of programming language used: Bythors is implemented in Rust and Lean 4, with the Lean components ultimately compiling down to C. On the other hand, vard is implemented in OCaml, with its protocol implementation having been extracted to OCaml from Coq. Writing well-optimised OCaml code is possible, but perhaps this may not have been the main focus of the authors: vard was created to showcase the performance of using an implementation of Raft verified with the library Verdi. Hence, assuming minimal optimisation effort, it is not uncommon for Rust and C code to generally outperform OCaml code.

We note that the benchmarks may not be fully conclusive as a dedicated testing setup was not employed. Nevertheless, we find it unlikely for the results to vary significantly more than an order of magnitude, so we claim that the results presented are still meaningful in demonstrating that running a verified protocol in a theorem-prover language is still practically feasible.

Chapter 5

Conclusions and Future Work

5.1 Overall Evaluation

Overall, we claim that it is practically feasible to interact directly with a verified implementation of a consensus protocol written in a theorem-prover language, and we demonstrate this by developing and benchmarking the Bythors replicated key-value store.

We posit that our approach of directly running the protocol in a theorem prover has benefits over extracting out an implementation, as we need to trust additional programs (which themselves may not be verifiably correct). With that said, we note that has recently been work done to develop a verified extraction mechanism for Coq [24], which perhaps invalidates most of the limitations we argue program extraction has. However, we note that our objective is not to discredit extraction as a poor method of making theorem prover programs practical, but rather to demonstrate that there may exist other feasible ways to do so.

5.2 Future work

5.2.1 Performance Optimsiations

As our project was concerned with developing a proof-of-concept replicated key-value store, writing well-optimised code was not in the scope of this project. We discuss here several inefficiencies in the code, and possible optimisations for them.

One notable inefficiency in our code is the significant overhead incurred when marshalling between Lean and Rust data representations. For example, the conversion of a Lean string to an owned Rust `String` currently involves an $O(n)$ copy of the string bytes (`src/marshal/string.rs`). However, this copying can be skipped if our implementation omits the use of owned `Strings` and uses string references (`&str`) instead.

Another inefficiency lies in the use of Lean `List`s. The Raft implementation in Lean uses linked lists as the primary array-like data structure. Similarly, association lists are used in place of hash maps when a dictionary-like data structure is needed. These data structures are useful for proof engineering as it is easier to prove properties on these data structures than arrays (`lib/Raft.lean:132`). However, these data structures are less performant than their array-based counterparts. In addition, when marshalling a Lean `List` to a Rust `Vec`, we currently perform an O(n) conversion to a Lean `Array`, before doing another O(n) element-wise copy to the Rust vector, which is particularly inefficient (`src/marshal/array.rs`).

Furthermore, whenever a packet is sent between nodes, that packet is currently first marshalled from Lean to Rust before being serialised and sent over the wire (`src/protocol/raft/protocol.rs:176`). However, we do not manipulate the packet on the Rust side apart from serialising it and sending it. Hence, directly serialising the packets from the Lean representation could significantly reduce marshalling overhead.

Finally, despite our careful attempts to maintain the proper reference counts when working with Lean objects in Rust, working over the FFI boundary is inherently unsafe and programmer errors may occur. Though testing has revealed that the code is unlikely to segmentation fault as a result from wrongly decrementing a reference-counted object, there is still the possibility of memory leaks arising from wrongly incrementing a reference-counted object, which is not something we have tested for.

References

[1] rocq/plugins/extraction at master · rocq-prover/rocq. URL https://github.com/rocq-prover/rocq/tree/master/plugins/extraction.

[2] etcd. URL https://etcd.io/.

[3] Code Generation Targets — Idris 1.3.3 documentation, . URL https://docs.idris-lang.org/en/latest/reference/codegen.html.

[4] Changes since Idris 1 - Idris2 0.0 documentation, . URL https://idris2.readthedocs.io/en/latest/updates/updates.html.

[5] Foreign Function Interface - Lean Documentation Overview, . URL https://lean-lang.org/lean4/doc/dev/ffi.html.

[6] What is Lean - Lean Documentation Overview, . URL https://lean-lang.org/lean4/doc/.

[7] The Polynomial Freiman-Ruzsa Conjecture. URL https://teorth.github.io/pfr//.

[8] FFI - The Rustonomicon. URL https://doc.rust-lang.org/nomicon/ffi.html.

[9] Tokio - An asynchronous Rust runtime, . URL https://tokio.rs/.

[10] tokio::task - Rust, . URL https://docs.rs/tokio/latest/tokio/task/index.html.

[11] A Byzantine failure in the real world, Nov. 2020. URL https://blog.cloudflare.com/a-byzantine-failure-in-the-real-world/.

[12] [July 2nd 2024] We have proved "BB(5) = 47,176,870", July 2024. URL https://discuss.bbchallenge.org/t/july-2nd-2024-we-have-proved-bb-5-47-176-870/237.

[13] leanprover/vscode-lean4, Apr. 2025. URL https://github.com/leanprover/vscode-lean4. original-date: 2020-12-31T17:26:54Z.

[14] d. . lean-sys, . URL https://github.com/digama0/lean-sys.

[15] f. . lean-sys (custom fork), . URL https://github.com/filbertphang/lean-sys.

[16] J. Avigad, L. De Moura, and S. Kong. Theorem proving in lean. 2021.

[17] E. Berkay Gulcan, B. Kulahcioglu Ozkan, R. Majumdar, and S. Nagendra. Model-guided fuzzing of distributed systems. *arXiv e-prints*, pages arXiv–2410, 2024.

[18] A. Bove, P. Dybjer, and U. Norell. A brief overview of agda — a functional language with dependent types. In *Proceedings of the 22nd International Conference on Theorem Proving in Higher Order Logics*, TPHOLs '09, pages 73–78, Berlin, Heidelberg, 2009. Springer-Verlag. ISBN 9783642033582. doi: 10.1007/978-3-642-03359-9_6. URL https://doi.org/10.1007/978-3-642-03359-9_6.

[19] W. Bugden and A. Alahmar. Rust: The programming language for safety and performance, 2022. URL https://arxiv.org/abs/2206.05503.

[20] C. Copeland and H. Zhong. Tangaroa: a byzantine fault tolerant raft. *Stanford University*, 2016.

[21] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair. *Distributed Systems: Concepts and Design*. Pearson Education, 2011. ISBN 9780133001372. URL https://books.google.com.sg/books?id=3ZouAAAAQBAJ.

[22] G. De Micheli, R. Ernst, and W. Wolf, editors. *Readings in hardware/software co-design*. Kluwer Academic Publishers, USA, 2001. ISBN 1558607021.

[23] P. Fonseca, K. Zhang, X. Wang, and A. Krishnamurthy. An empirical study on the correctness of formally verified distributed systems. In *Proceedings of the Twelfth European Conference on Computer Systems*, EuroSys '17, pages 328–343, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450349383. doi: 10.1145/3064176.3064183. URL https://doi.org/10.1145/3064176.3064183.

[24] Y. Forster, M. Sozeau, and N. Tabareau. Verified Extraction from Coq to OCaml. *Proceedings of the ACM on Programming Languages*, 8(PLDI):52–75, June 2024. doi: 10.1145/3656379. URL https://inria.hal.science/hal-04329663.

[25] T. I. Foundation. libp2p - A modular network stack. URL https://libp2p.io/.

[26] G. Gonthier. The four colour theorem: Engineering of a formal proof. In *Computer Mathematics: 8th Asian Symposium, ASCM 2007, Singapore, December 15-17,*

2007. Revised and Invited Papers, page 333, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 978-3-540-87826-1. doi: 10.1007/978-3-540-87827-8_28. URL https://doi.org/10.1007/978-3-540-87827-8_28.

[27] R. Hähnle. *A New Look at Formal Methods for Software Construction*, pages 1–18. 04 2007. ISBN 978-3-540-68977-5. doi: 10.1007/978-3-540-69061-0_1.

[28] C. Jensen, H. Howard, and R. Mortier. Examining raft's behaviour during partial network failures. In *Proceedings of the 1st Workshop on High Availability and Observability of Cloud Systems*, HAOC '21, page 11–17, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383363. doi: 10.1145/3447851.3458739. URL https://doi.org/10.1145/3447851.3458739.

[29] C. Kern and M. R. Greenstreet. Formal verification in hardware design: a survey. *ACM Trans. Des. Autom. Electron. Syst.*, 4(2):123–193, Apr. 1999. ISSN 1084-4309. doi: 10.1145/307988.307989. URL https://doi.org/10.1145/307988.307989.

[30] V. Lenko, V. Pasichnyk, N. Kunanets, and Y. Shcherbyna. Type- theoretical foundations of the derivation system in coq. In *2018 IEEE First International Conference on System Analysis & Intelligent Computing (SAIC)*, pages 1–6, 2018. doi: 10.1109/SAIC.2018.8516885.

[31] X. Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, July 2009. ISSN 0001-0782. doi: 10.1145/1538788.1538814. URL https://doi.org/10.1145/1538788.1538814.

[32] P. Letouzey. Extraction in coq, an overview. 06 2008. doi: 10.1007/978-3-540-69407-6.

[33] F. Maric. A survey of interactive theorem proving. *Zbornik radova*, 18(26):173–223, 2015.

[34] K. L. McMillan and O. Padon. Ivy: A multi-modal verification tool for distributed algorithms. *Computer Aided Verification*, 12225:190 – 202, 2020. URL https://api.semanticscholar.org/CorpusID:220547714.

[35] L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction - CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings*, pages 625–635, Berlin, Heidelberg, 2021. Springer-Verlag. ISBN 978-3-030-79875-8. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.

[36] H. S. Nalawala, J. Shah, S. Agrawal, and P. Oza. A comprehensive study of "etcd"—an open-source distributed key-value store with relevant distributed databases. In *Emerging Technologies for Computing, Communication and Smart Cities: Proceedings of ETCCS 2021*, pages 481–489. Springer, 2022.

[37] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX annual technical conference (USENIX ATC 14)*, pages 305–319, 2014.

[38] F. B. Schneider. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Comput. Surv.*, 22(4):299–319, Dec. 1990. ISSN 0360-0300. doi: 10.1145/98163.98167. URL https://doi.org/10.1145/98163.98167.

[39] Stankovic. A perspective on distributed computer systems. *IEEE Transactions on Computers*, C-33(12):1102–1115, 1984. doi: 10.1109/TC.1984.1676389.

[40] M. Taube, G. Losa, K. L. McMillan, O. Padon, M. Sagiv, S. Shoham, J. R. Wilcox, and D. Woos. Modularity for decidability of deductive verification with applications to distributed systems. *SIGPLAN Not.*, 53(4):662–677, June 2018. ISSN 0362-1340. doi: 10.1145/3296979.3192414. URL https://doi.org/10.1145/3296979.3192414.

[41] D. Thinkers. Raft does not Guarantee Liveness in the face of Network Faults. URL https://decentralizedthoughts.github.io/2020-12-12-raft-liveness-full-omission/.

[42] A. Wahab and W. Mehmood. Survey of consensus protocols, 2018. URL https://arxiv.org/abs/1810.03357.

[43] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson. Verdi: a framework for implementing and formally verifying distributed systems. *SIGPLAN Not.*, 50(6):357–368, June 2015. ISSN 0362-1340. doi: 10.1145/2813885.2737958. URL https://doi.org/10.1145/2813885.2737958.

[44] D. Woos, J. R. Wilcox, S. Anton, Z. Tatlock, M. D. Ernst, and T. Anderson. Planning for change in a formal verification of the raft consensus protocol. In *Proceedings of the 5th ACM SIGPLAN Conference on Certified Programs and Proofs*, CPP 2016, page 154–165, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450341271. doi: 10.1145/2854065.2854081. URL https://doi.org/10.1145/2854065.2854081.